1. Project Overview

A full stack online gaming portal named Romlex with a dynamic grid layout, playable demo iframes, user profiles, and a secure admin dashboard. The site features a direct chat based inquiry system for premium licenses. Packaged with Docker for easy local setup.

2. Goal of the Website

- Showcase web based games in an interactive grid
- Provide a chat based license inquiry system, no payment gateway
- Keep the site fast and easy to navigate
- Allow the owner to manage game listings directly through an admin panel
- Be fully responsive on mobile phones and desktop browsers
- Be ready to run locally using a single Docker command

3. Page Structure

Section 1: Home Page

- Brand name (Romlex) and search bar at the top
- Hero slider displaying featured games
- Category sidebar (Action, Puzzle, Strategy)
- Main grid showing game cards
- Game cards show thumbnail, title, and category

Section 2: Game Playback View

- Embedded iframe for playing the demo game
- Sidebar with publisher info, player ratings, and view count
- Add to Favorites button
- Inquire for License via chat button

Section 3: User Dashboard

- Registration and login screens
- History tab showing recently played games
- Favorites tab showing saved games
- Profile section to update username and avatar

4. Admin Panel

- Secure login restricted to the site owner
- Form to add new games (title, category, iframe URL, and WebP thumbnail upload)
- List view to edit existing games or delete them
- Toggle switch to make a game active or hidden on the frontend

5. Chat Integration (Only Inquiry Method)

- Inquire for License Button (on the game playback page)
- Opens a direct chat link (like wa.me) with a pre filled message
- Example pre filled message: Hi! I want to inquire about a license for the game [Game Title] on Romlex. Please share details.
- Desktop fallback: If the chat app is not installed, show a modal with the contact number and a Copy Number button.

6. Animations (Keep It Simple)

All animations should be lightweight and CSS based.

- Allowed Tools: Pure CSS animations and transitions, AOS library for scroll effects.
- Required Animations: Game card hover lift and shadow, hero slider CSS transitions, sections fade in on scroll.
- Accessibility: Respect prefers-reduced-motion CSS media query.

7. Database and Data Structure

- Relational database using SQLite or PostgreSQL
- The backend must read and write to the database dynamically
- Tables needed: Users, Games, Categories, User_Favorites, User_History

8. SEO Basics

- Page title and meta description tags filled with brand information
- Open Graph tags
- Semantic HTML tags used throughout
- A simple sitemap.xml and robots.txt in the root folder

9. Accessibility Basics

- Alt text on every image
- ARIA labels on icon only buttons
- All buttons and links should be keyboard navigable using Tab key
- Decent color contrast between text and background

10. Image and Asset Specifications

- All game thumbnails optimized as WebP format and stay under 200KB per image
- Lazy loading enabled on all game grid images
- Favicon included in standard sizes

11. Tech Stack and Code Standards

- Backend: Python (FastAPI or Flask)
- Frontend: HTML, CSS (Tailwind or Bootstrap allowed), vanilla JavaScript
- Database: SQLite or PostgreSQL
- No heavy JS frameworks like React or Angular
- Mobile first responsive design
- Lighthouse performance score of at least 85
- Tested on Chrome, Safari, Firefox, and Edge
- Code should follow standard indentation conventions and be well commented

12. Docker Setup Requirements

- Dockerfile using Alpine (Nginx for static frontend files, Python environment for backend)
- docker compose yml file for one command startup
- Default port 8080, README explains how to change it
- Run the site with: docker compose up

13. Folder Structure

- project-root/
- backend/(app.py, models.py, requirements.txt)
- frontend/(index.html, css/, js/)
- database/ (schema.sql or db file)

- assets/(images, icons)
- Dockerfile
- docker-compose.yml
- README.md

14. Required Deliverables

- Complete application files strictly organized into backend, frontend, and database folders
- Seed file for the database with sample data for at least 10 free demo games
- Dockerfile and docker compose yml
- Favicon files
- sitemap.xml and robots.txt
- README md file with setup, update, and deployment instructions

15. What NOT to Do

- Do not use WordPress, Shopify, Wix, or any CMS platform
- Do not add any payment gateway, UPI integration, or automated checkout system
- Do not skip the Docker setup
- Do not skip mobile responsiveness
- Do not deliver code without comments or with poor folder structure